



Universidade de Brasília

Departamento de Ciência da Computação

VHDL Básico



Entidade

- Descreve o nome e as entradas e saídas dos módulos

```
entity somador is
    port (
        A, B : in std_logic_vector(7 downto 0);
        S      : out std_logic_vector(7 downto 0)
    );
end entity;
```



Arquitetura

- Descreve o comportamento ou estrutura do módulo

```
architecture rtl of somador is
begin
    S <= A + B;
end architecture;
```



Lógica Combinacional

- Saídas definidas pelos valores nas entradas apenas, sem estados internos.
 - Somadores
 - Multiplexadores
 - Decodificadores
 - Operadores lógicos
 - ULA



Lógica Combinacional

■ Comando WHEN-ELSE

- Comando concorrente, no corpo da arquitetura

```
Y <= A when SEL='0'  
     else B;
```

```
Y <=  A when SEL="00" else  
     B when SEL="01" else  
     C when SEL="10" else  
     "00000000";
```



Lógica Combinacional

- Comando WITH-SELECT
 - Comando concorrente, no corpo da arquitetura

```
with SEL select
    Y <= A when "00",
        B when "01",
        C when "10",
        D when others;
```



Lógica Combinacional

■ Processo Combinacional

- Um processo define um escopo com comandos sequenciais - comandos avaliados sequencialmente

```
process (A, B, C)
begin
    Y <= (A and B) or C;
end process;
```



Lógica Combinacional

■ Processo Combinacional

□ IF - THEN - ELSE

```
process (A,B,SEL)
begin
    if SEL='0' then
        Y <= A;
    else
        Y <= B;
    end if;
end process;
```



Lógica Combinacional

■ Processo Combinacional

□ CASE - WHEN

```
case OPCODE is
  when "00" => Y <= A+B;
  when "01" => Y <= A-B;
  when "10" => Y <= A and B;
  when others => Y <= (others=>'0');
end case;
```



β

■ Evitar latches

□ IF - THEN - ELSE incompleto

```
process(cod, ent)
begin
  if (cond = '1')
    then sai <= ent;
    -- Erro: Falta o 'else'. E se con for '0'?
    -- O circuito guarda o valor anterior (Latch).
  end if;
end process;
```



Lógica Combinacional

■ Evitar latches

□ IF - THEN - ELSE incompleto

```
process(cod, ent)
begin
    if (cond = '1')
        then sai <= ent;
        else sai <= not(ent);
    end if;
end process;
```

```
process(cod, ent)
begin
    sai <= not(ent);
    if (cond = '1')
        then sai <= ent;
    end if;
end process;
```



Lógica Combinacional

■ Evitar latches

□ CASE - WHEN incompleto

```
process(sel, ent)
begin
  case sel is
    when "00" => saida <= ent;
    when "01" => saida <= not ent;
    -- Erro: e se sel for "10" ou "11"?
    -- ou meta-valores (como 'X', 'U', 'Z')?
  end case;
end process;
```



Lógica Combinacional

■ Evitar latches

□ CASE - WHEN incompleto

```
process(sel, ent)
begin
  case sel is
    when "00" => saida <= ent;
    when "01" => saida <= not ent;
    when others => saida <= (others <= '0');
  end case;
end process;
```



Registradores

- Armazena dado na subida do relógio:

```
process (clk)
begin
    if rising_edge (clk) then
        Q <= D;
    end if;
end process;
```



Registadores

■ Com *reset* assíncrono:

```
process (clk, reset)
begin
    if reset='1' then
        Q <= (others=>'0');

    elsif rising_edge(clk) then
        Q <= D;
    end if;
end process;
```



Registadores

■ Com *reset* síncrono:

```
process (clk, reset)
begin
    if rising_edge(clk) then
        if reset='1' then
            Q <= (others=>'0');
        else Q <= D;
        end if;
    end process;
```



Registadores

■ Com habilitação de escrita:

```
process (clk)
begin
    if rising_edge (clk) then

        if WEN='1' then
            Q <= D;
        end if;

    end if;
end process;
```



Banco de Registradores

- Exemplo : 16 registradores de 16 bits
 - Declaração:

```
type reg_array is array(0 to 15)
    of std_logic_vector(15 downto 0);

signal breg : reg_array;
```



Banco de Registradores

■ Escrita no banco:

- wradd: índice do registrador a ser escrito
- we: write enable

```
process (clk)
begin
    if rising_edge(clk) then

        if we='1' then
            breg(wradd) <= WR_DATA;
        end if;

    end if;
end process;
```



Banco de Registradores

- Leitura do banco:
 - radd: índice do registrador a ser lido
 - rdata: saída do banco

```
process (clk)
begin
    rdata <= breg(radd);
end process;
```



Testbench

■ Estrutura:

- Instância do módulo (*DUT - Device under Test*)
- Geração de clock
- Geração de estímulos
- Verificação dos resultados



Declara DUT

```
entity tb is
end entity;

architecture a of tb is
    signal clk : std_logic := '0';
    signal a, b : std_logic_vector (15 downto 0);
    signal s : std_logic_vector (15 downto 0);
    component somador is
        port (
            a, b: in std_logic_vector (15 downto 0);
            s : out std_logic_vector (15 downto 0))
    end component;

begin
    ...
end;
```



Instancia DUT

```
entity tb is
end entity;

architecture a of tb is
    ...

begin
    dut: somador
        port map (
            a => a,
            b => b,
            s => s
        );
    ...
end;
```



Geração de clock

```
entity tb is
end entity;

architecture a of tb is
    ...

begin
    dut: ...

    clk <= not clk after 5 ns;

end;
```



Geração de Estímulos

```
process
begin

    A <= x"05";
    B <= x"03";

    wait for 100 ns;

    A <= x"10";
    B <= x"20";

    wait;

end process;
```



Assert

- Verificando os resultados com assert:

```
process
begin
    A <= x"05";
    B <= x"03";
    wait for 100 ns;
    assert s = x"08"
        report "Erro somador"
        severity error;
    wait;
end process;
```